GlowTap

TAP • BOOK • GLOW

UNISEX SALON & WELLNESS

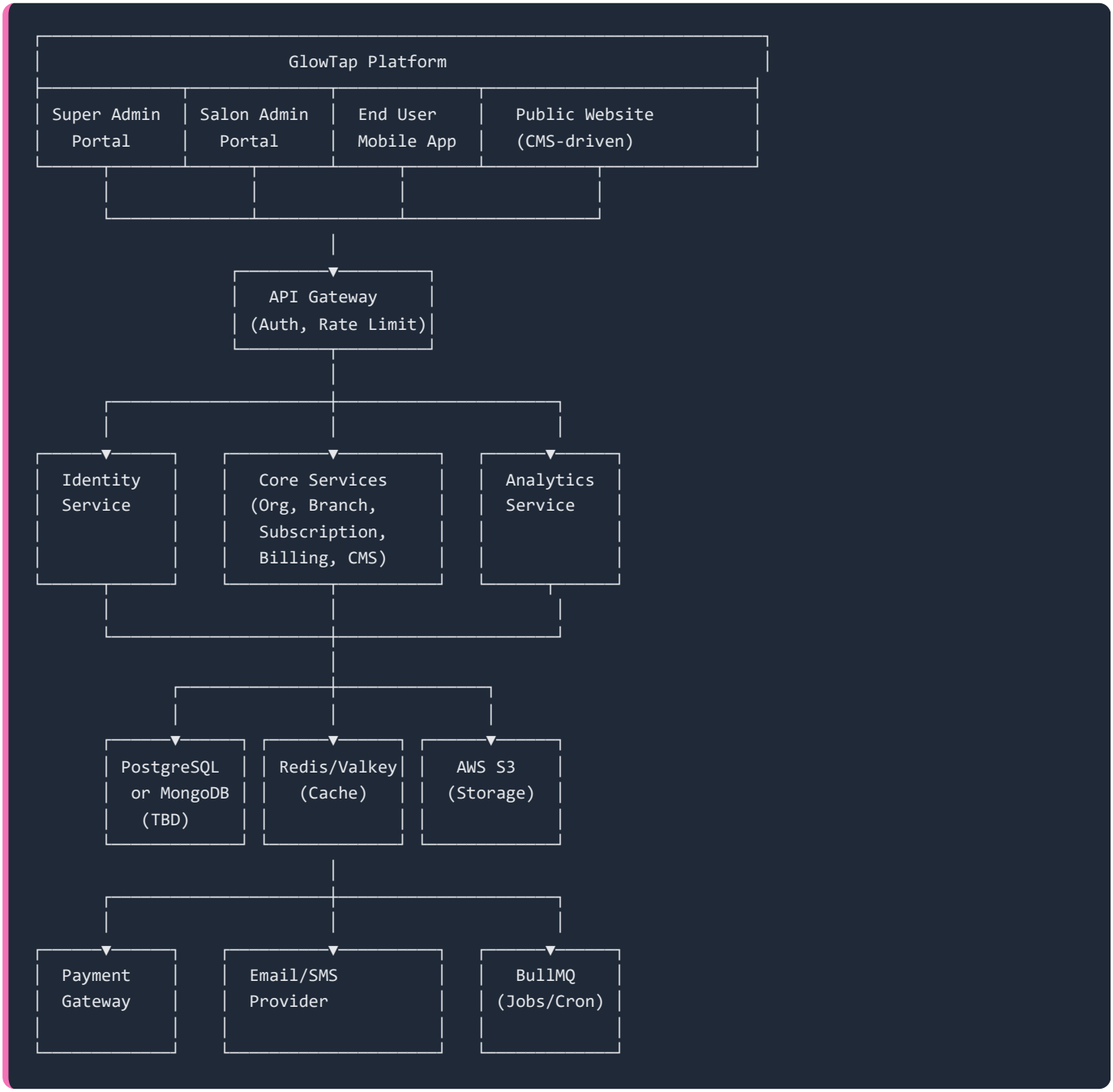
Solutioning & Technical Architecture

Super Admin Portal — Engineering & Implementation

Version 1.2 | **Date** May 31, 2026 | **Confidential**

1. System Context

1.1 High-Level Architecture



1.2 Technology Stack

Layer	Technology	Notes
Frontend (Web)	React + TypeScript	Super Admin Portal, Salon Admin Portal, public website
Frontend (Mobile)	React Native + TypeScript	End-user app, optional salon admin mobile
State Management	Redux Toolkit + RTK Query	Shared patterns across web and mobile; RTK Query for API caching, mutations, and invalidation
Backend API	Node.js + Fastify (TypeScript)	High-performance HTTP API; modular plugin architecture
Database	PostgreSQL **or** MongoDB *(TBD — Yash)*	PostgreSQL preferred for relational billing/audit; MongoDB viable for flexible CMS/banner config. Schema examples in this doc use PostgreSQL/SQL; MongoDB equivalent collections noted where relevant
Cache	Redis / Valkey	Sessions, banner rules, rate limiting, BullMQ backing store
File Storage	AWS S3	CMS media, KYC documents, invoice PDFs, report exports
Background Jobs	BullMQ	Daily defaulter reports, email dispatch, webhooks, approval pipeline async tasks
Logging	Pino → cloud *(provider TBD — Yash)*	Structured JSON logs shipped to cloud log aggregator
Auth	JWT + Refresh tokens, MFA (TOTP)	Stateless API auth; sessions cached in Redis/Valkey
Deployment	AWS EC2 (production)	Application servers for API + worker processes; S3 for static assets
PDF/Export	Server-side CSV/Excel generation	Report exports stored in S3

Frontend — Redux Toolkit + RTK Query

```
┌──────────────────────────────────────────────────┐
│  React / React Native UI                         │
├──────────────────────────────────────────────────┤
│  Redux Toolkit (slices)                          │
│    auth, ui, filters, notifications              │
├──────────────────────────────────────────────────┤
│  RTK Query (api slice)                           │
│    baseQuery → Fastify API (/api/v1)             │
│    tag-based cache invalidation per resource     │
│    endpoints: organizations, branches, plans, tickets…  │
└──────────────────────────────────────────────────┘
```

- **Shared API layer:** RTK Query `createApi` definitions can be shared between web and mobile where endpoints overlap
- **Cache tags:** e.g. `Organization`, `Branch`, `Subscription`, `Ticket` — invalidated on approve/suspend/mutation
- **Optimistic updates:** For ticket status changes, banner toggles

Backend — Fastify (TypeScript)

```
src/
├── app.ts                  # Fastify instance, plugin registration
├── plugins/
│   ├── auth.ts             # JWT verification hook
│   ├── rbac.ts             # Role-based access control
│   └── pino-logger.ts      # Structured logging
├── modules/
│   ├── organizations/
│   ├── branches/
│   ├── subscriptions/
│   ├── transactions/
│   ├── banners/
│   ├── tickets/
│   └── cms/
├── jobs/                   # BullMQ workers
│   ├── defaulter-report.worker.ts
│   ├── email.worker.ts
│   └── approval-pipeline.worker.ts
├── lib/
│   ├── db/                 # PostgreSQL (Drizzle/Prisma) or MongoDB (Mongoose)
│   ├── cache/              # Redis/Valkey client
│   └── s3/                 # AWS S3 client
└── types/
```

- **Validation:** `@fastify/type-provider-typebox` or Zod schemas for request/response
- **OpenAPI:** `@fastify/swagger` for auto-generated API docs
- **Workers:** Separate EC2 process (or same instance, different PM2 entry) for BullMQ consumers

---

© 2026 GlowTap

2. Domain Model

2.1 Entity Relationship Overview

```
Organization (Salon)
├── has many Branches
├── has one active Subscription
├── has many SubscriptionTransactions
├── has commission_pct (per-salon rate)
├── has many EndUserTransactions (with commission_amount)
├── has one SalonAdmin (primary)
├── has many SubAdmins (via salon portal)
└── earnings = subscription revenue + commission revenue

SubscriptionPlan
├── defines max_branches
├── defines feature_entitlements (JSONB)
├── has many PlanDiscounts (contract-length discounts)
└── has many OrganizationSubscriptions

ReasonCode
├── category: SUSPEND | TERMINATE | REJECT | TICKET
├── linked pricing_penalty (optional)
└── used in AuditLog, Tickets, SalonActions

Banner
├── targeting_rules (JSONB)
├── display_priority
└── active_date_range

SupportTicket
├── reason_code_id
├── sub_reason_code_id
├── status workflow
└── linked org/branch/user (optional)
```

2.2 Core Tables

> **Note:** Schema below assumes **PostgreSQL**. If **MongoDB** is chosen, equivalent collections with embedded documents for `address`, `feature_entitlements`, `targeting_rules`, and `kyc_documents` replace normalized tables. Foreign-key relationships become `ObjectId` references with application-level consistency.

organizations

```
CREATE TABLE organizations (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name            VARCHAR(255) NOT NULL,
  slug            VARCHAR(100) UNIQUE NOT NULL,
  owner_name      VARCHAR(255) NOT NULL,
  owner_email     VARCHAR(255) NOT NULL,
  owner_phone     VARCHAR(20),
  gst_number      VARCHAR(50),
  logo_url        TEXT,
  registered_address JSONB,
  status          VARCHAR(20) NOT NULL DEFAULT 'PENDING'
                  CHECK (status IN ('PENDING','ACTIVE','SUSPENDED','TERMINATED')),
  kyc_status      VARCHAR(20) DEFAULT 'NOT_REQUIRED',
  kyc_documents   JSONB,
  commission_pct  DECIMAL(5,2) NOT NULL DEFAULT 5.00,
  commission_pct_updated_at TIMESTAMPTZ,
  commission_pct_updated_by UUID REFERENCES super_admin_users(id),
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW(),
  approved_at     TIMESTAMPTZ,
  approved_by     UUID REFERENCES super_admin_users(id),
  terminated_at   TIMESTAMPTZ,
  termination_reason_id UUID REFERENCES reason_codes(id)
);
```

branches

```
CREATE TABLE branches (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES organizations(id),
  name            VARCHAR(255) NOT NULL,
  address         JSONB NOT NULL,
  contact_phone   VARCHAR(20),
  operating_hours JSONB,
  status          VARCHAR(20) NOT NULL DEFAULT 'PENDING'
                  CHECK (status IN ('PENDING','ACTIVE','INACTIVE','REJECTED')),
  is_primary      BOOLEAN DEFAULT FALSE,
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  approved_at     TIMESTAMPTZ,
  approved_by     UUID REFERENCES super_admin_users(id),
  rejection_reason_id UUID REFERENCES reason_codes(id)
);

CREATE INDEX idx_branches_org ON branches(organization_id);
CREATE INDEX idx_branches_status ON branches(status);
```

© 2026 GlowTap

Confidential

subscription_plans

```
CREATE TABLE subscription_plans (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code            VARCHAR(50) UNIQUE NOT NULL,
  name            VARCHAR(100) NOT NULL,
  duration_months INT NOT NULL,
  base_price      DECIMAL(12,2) NOT NULL,
  recurring_price DECIMAL(12,2) DEFAULT 0,
  max_branches    INT NOT NULL,
  feature_entitlements JSONB NOT NULL DEFAULT '{}',
  display_description TEXT,
  status          VARCHAR(20) DEFAULT 'ACTIVE'
                  CHECK (status IN ('ACTIVE','DEPRECATED','RETIRED')),
  sort_order      INT DEFAULT 0,
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW()
);
```

organization_subscriptions

```
CREATE TABLE organization_subscriptions (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES organizations(id),
  plan_id         UUID NOT NULL REFERENCES subscription_plans(id),
  status          VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
                  CHECK (status IN ('ACTIVE','PAST_DUE','CANCELLED','EXPIRED')),
  start_date      DATE NOT NULL,
  end_date        DATE NOT NULL,
  next_bill_date  DATE,
  auto_renew      BOOLEAN DEFAULT TRUE,
  branch_limit_override INT,
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW()
);
```

© 2026 GlowTap

Confidential

subscription_discounts

```
CREATE TABLE subscription_discounts (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES organizations(id),
  discount_type   VARCHAR(3) NOT NULL CHECK (discount_type IN ('RC','NRC')),
  amount_type     VARCHAR(10) NOT NULL CHECK (amount_type IN ('FIXED','PERCENT')),
  amount          DECIMAL(12,2) NOT NULL,
  description     TEXT NOT NULL,
  effective_date  DATE NOT NULL,
  expiry_date     DATE,
  applied_by      UUID REFERENCES super_admin_users(id),
  applied_at      TIMESTAMPTZ DEFAULT NOW(),
  is_applied      BOOLEAN DEFAULT FALSE,
  invoice_id      UUID
);
```

plan_contract_discounts

```
CREATE TABLE plan_contract_discounts (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  plan_id         UUID NOT NULL REFERENCES subscription_plans(id),
  contract_months INT NOT NULL,
  discount_type   VARCHAR(10) NOT NULL,
  discount_value  DECIMAL(12,2) NOT NULL,
  display_text    TEXT NOT NULL,
  valid_from      DATE,
  valid_until     DATE,
  is_active       BOOLEAN DEFAULT TRUE
);
```

reason_codes

```
CREATE TABLE reason_codes (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  category        VARCHAR(20) NOT NULL
                  CHECK (category IN ('SUSPEND','TERMINATE','REJECT','TICKET','TICKET_SUB')),
  code            VARCHAR(50) UNIQUE NOT NULL,
  label           VARCHAR(255) NOT NULL,
  description     TEXT,
  parent_id       UUID REFERENCES reason_codes(id),
  pricing_impact  JSONB,
  is_active       BOOLEAN DEFAULT TRUE,
  sort_order      INT DEFAULT 0
);
```

© 2026 GlowTap

Confidential

end_user_transactions

```
CREATE TABLE end_user_transactions (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_ref VARCHAR(100) UNIQUE NOT NULL,
  end_user_id     UUID NOT NULL REFERENCES end_users(id),
  organization_id UUID NOT NULL REFERENCES organizations(id),
  branch_id       UUID NOT NULL REFERENCES branches(id),
  gross_amount    DECIMAL(12,2) NOT NULL,
  commission_pct  DECIMAL(5,2) NOT NULL,
  commission_amount DECIMAL(12,2) NOT NULL,
  net_to_salon    DECIMAL(12,2) NOT NULL,
  currency        VARCHAR(3) DEFAULT 'INR',
  payment_method  VARCHAR(50),
  gateway_ref     VARCHAR(255),
  status          VARCHAR(20) NOT NULL
                  CHECK (status IN ('SUCCESS','FAILED','REFUNDED','DISPUTED','PENDING')),
  service_details JSONB,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_eut_org ON end_user_transactions(organization_id);
CREATE INDEX idx_eut_branch ON end_user_transactions(branch_id);
CREATE INDEX idx_eut_created ON end_user_transactions(created_at);
CREATE INDEX idx_eut_status ON end_user_transactions(status);
```

commission_rate_history

Audit trail for commission percentage changes per salon.

```
CREATE TABLE commission_rate_history (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES organizations(id),
  previous_pct    DECIMAL(5,2) NOT NULL,
  new_pct         DECIMAL(5,2) NOT NULL,
  effective_from  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  changed_by      UUID NOT NULL REFERENCES super_admin_users(id),
  reason          TEXT,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

platform_commission_config

Global defaults for commission model.

```
CREATE TABLE platform_commission_config (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  default_pct     DECIMAL(5,2) NOT NULL DEFAULT 5.00,
  min_pct         DECIMAL(5,2) NOT NULL DEFAULT 1.00,
  max_pct         DECIMAL(5,2) NOT NULL DEFAULT 25.00,
  updated_at      TIMESTAMPTZ DEFAULT NOW()
);
```

subscription_transactions

```
CREATE TABLE subscription_transactions (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES organizations(id),
  subscription_id UUID REFERENCES organization_subscriptions(id),
  invoice_number  VARCHAR(50) UNIQUE,
  amount          DECIMAL(12,2) NOT NULL,
  discount_amount DECIMAL(12,2) DEFAULT 0,
  net_amount      DECIMAL(12,2) NOT NULL,
  transaction_type VARCHAR(20) NOT NULL
                  CHECK (transaction_type IN ('PURCHASE','RENEWAL','UPGRADE','REFUND','CREDIT')),
  payment_status  VARCHAR(20) NOT NULL,
  gateway_ref     VARCHAR(255),
  created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

banners

```
CREATE TABLE banners (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title           VARCHAR(255) NOT NULL,
  image_url       TEXT,
  cta_text        VARCHAR(100),
  cta_link        TEXT,
  targeting_rules JSONB NOT NULL,
  display_priority INT DEFAULT 0,
  active_from     TIMESTAMPTZ NOT NULL,
  active_until    TIMESTAMPTZ,
  is_active       BOOLEAN DEFAULT TRUE,
  created_by      UUID REFERENCES super_admin_users(id),
  created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

© 2026 GlowTap

Confidential

support_tickets

```
CREATE TABLE support_tickets (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ticket_number   VARCHAR(20) UNIQUE NOT NULL,
  reason_id       UUID NOT NULL REFERENCES reason_codes(id),
  sub_reason_id   UUID REFERENCES reason_codes(id),
  description     TEXT NOT NULL,
  status          VARCHAR(30) NOT NULL DEFAULT 'CREATED'
                  CHECK (status IN ('CREATED','IN_PROGRESS','ON_HOLD',
                                    'AWAITING_USER_CONFIRMATION','RESOLVED')),
  priority        VARCHAR(10) DEFAULT 'MEDIUM',
  source          VARCHAR(20) DEFAULT 'SUPER_ADMIN',
  organization_id UUID REFERENCES organizations(id),
  branch_id       UUID REFERENCES branches(id),
  end_user_id     UUID REFERENCES end_users(id),
  assigned_to     UUID REFERENCES super_admin_users(id),
  created_by      UUID,
  resolved_at     TIMESTAMPTZ,
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW()
);
```

audit_logs

```
CREATE TABLE audit_logs (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_id        UUID NOT NULL,
  actor_type      VARCHAR(20) NOT NULL,
  action          VARCHAR(100) NOT NULL,
  entity_type     VARCHAR(50) NOT NULL,
  entity_id       UUID NOT NULL,
  before_state    JSONB,
  after_state     JSONB,
  ip_address      INET,
  session_id      VARCHAR(100),
  created_at      TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_audit_entity ON audit_logs(entity_type, entity_id);
CREATE INDEX idx_audit_actor ON audit_logs(actor_id);
CREATE INDEX idx_audit_created ON audit_logs(created_at);
```

3. Subscription Plan & Branch Limit Design

3.1 Branch Limit Enforcement

Branch limits are enforced at **three layers**:

```
┌─────────────────────────────────────────────────────┐
│ Layer 1: API Validation (Branch Create Request)     │
│   IF active_branch_count >= effective_max_branches  │
│   THEN reject with UPGRADE_REQUIRED error           │
├─────────────────────────────────────────────────────┤
│ Layer 2: Super Admin Approval Gate                  │
│   Branch approval UI shows limit status             │
│   Warn if at/over limit; require plan change first  │
├─────────────────────────────────────────────────────┤
│ Layer 3: Database Constraint (soft)                 │
│   Trigger or application check on branch status → ACTIVE│
└─────────────────────────────────────────────────────┘
```

Effective max branches calculation:

```
effective_max = COALESCE(
  organization_subscriptions.branch_limit_override,
  subscription_plans.max_branches
)
```

3.2 Plan Change & Branch Reconciliation

Scenario	Behavior
Upgrade (more branches allowed)	Immediate; new branches can be requested
Downgrade (fewer branches allowed)	Block if `active_branch_count > new_max`; salon must deactivate branches first
Plan expiry	Grace period (configurable); branches remain active until suspension
Override by Super Admin	Set `branch_limit_override` on subscription with audit log

3.3 Recommended Plan-Branch Matrix (Configurable)

```
{
  "STARTER_1M":  { "max_branches": 1,  "features": ["appointments", "services"] },
  "GROWTH_6M":   { "max_branches": 3,  "features": ["appointments", "services", "manpower"] },
  "PRO_12M":     { "max_branches": 10, "features": ["appointments", "services", "manpower", "analytics_ba
  "ENTERPRISE_24M": { "max_branches": -1, "features": ["all"] }
}
```

> `-1` represents unlimited; enforce reasonable soft cap (e.g., 100) at API level.

3.4 Commission & Earnings Model

GlowTap revenue = **Subscription revenue** + **Commission revenue**.

Commission Calculation (on payment success)

```
function calculateCommission(
  grossAmount: number,
  commissionPct: number
): { commissionAmount: number; netToSalon: number } {
  const commissionAmount = round(grossAmount * (commissionPct / 100), 2);
  const netToSalon = round(grossAmount - commissionAmount, 2);
  return { commissionAmount, netToSalon };
}
```

- `commission_pct` read from `organizations.commission_pct` at transaction time
- Stored on `end_user_transactions` (snapshot — not recalculated if rate changes later)
- On refund: reverse `commission_amount` proportionally

Salon Earnings Aggregation

```
-- GET /organizations/:id/earnings
SELECT
  o.id,
  o.commission_pct AS current_commission_pct,
  COALESCE(sub.total, 0) AS subscription_revenue,
  COALESCE(com.total, 0) AS commission_revenue,
  COALESCE(sub.total, 0) + COALESCE(com.total, 0) AS total_glowtap_earnings
FROM organizations o
LEFT JOIN (
  SELECT organization_id, SUM(net_amount) AS total
  FROM subscription_transactions
  WHERE payment_status = 'SUCCESS'
  GROUP BY organization_id
) sub ON sub.organization_id = o.id
LEFT JOIN (
  SELECT organization_id, SUM(commission_amount) AS total
  FROM end_user_transactions
  WHERE status = 'SUCCESS'
  GROUP BY organization_id
) com ON com.organization_id = o.id
WHERE o.id = :org_id;
```

Commission Rate Update

```
PATCH /organizations/:id/commission
  → validate min_pct <= commission_pct <= max_pct
  → update organizations.commission_pct
  → insert commission_rate_history
  → audit_log entry
  → does NOT retroactively update past transactions
```

© 2026 GlowTap

Confidential

4.1 API Conventions

- Base path: `/api/v1/super-admin`
- Authentication: Bearer JWT in `Authorization` header
- Pagination: `?page=1&limit=20`
- Sorting: `?sort=created_at&order=desc`
- Filtering: query params per resource
- Response envelope:

```
{
  "success": true,
  "data": {},
  "meta": { "page": 1, "limit": 20, "total": 150 }
}
```

4.2 Key API Endpoints

Authentication

Method	Endpoint	Description
POST	`/auth/login`	Email + password login
POST	`/auth/mfa/verify`	TOTP verification
POST	`/auth/refresh`	Refresh access token
POST	`/auth/logout`	Invalidate session

Salon Approvals

Method	Endpoint	Description
GET	`/leads`	List pending salon leads
GET	`/leads/:id`	Lead detail with documents
POST	`/leads/:id/approve`	Approve with `{ commission_pct }` and trigger onboarding pipeline
POST	`/leads/:id/reject`	Reject with reason_code_id
POST	`/leads/:id/request-info`	Request additional documents

Approve Pipeline (async job):

```
1. Create organization (status: ACTIVE, commission_pct from approval payload)
2. Create primary branch (status: ACTIVE)
3. Create organization_subscription
4. Create salon_admin user (Identity Service)
5. Assign all branch permissions
6. Send welcome email with credentials
7. Write audit_log entries
8. Emit event: salon.approved
```

Branch Approvals

Method	Endpoint	Description
GET	`/branch-requests`	List pending branch requests
GET	`/branch-requests/:id`	Detail with org subscription info
POST	`/branch-requests/:id/approve`	Approve branch
POST	`/branch-requests/:id/reject`	Reject with reason

Organizations (All Salons)

Method	Endpoint	Description
GET	`/organizations`	List with filters (includes commission_pct, total_earnings)
GET	`/organizations/:id`	Full org detail
GET	`/organizations/:id/earnings`	Earnings summary: subscription + commission, with date range
GET	`/organizations/:id/earnings/subscription`	Subscription payment line items
GET	`/organizations/:id/earnings/commission`	Commission line items per transaction
GET	`/organizations/:id/earnings/export`	CSV/PDF export
PATCH	`/organizations/:id`	Update profile fields
PATCH	`/organizations/:id/commission`	`{ commission_pct, reason?, effective_from? }`
GET	`/organizations/:id/commission/history`	Commission rate change audit trail
POST	`/organizations/:id/suspend`	`{ reason_code_id, notes }`
POST	`/organizations/:id/resume`	`{ notes }`
POST	`/organizations/:id/terminate`	`{ reason_code_id, notes }`
POST	`/organizations/:id/subscription/change-plan`	`{ plan_id, effective_date }`
POST	`/organizations/:id/subscription/discount`	`{ type, amount_type, amount, description, effective_date }`

Transactions

Method	Endpoint	Description
GET	`/transactions/end-user`	B2C transactions with filters
GET	`/transactions/end-user/:id`	Transaction detail
GET	`/transactions/subscription`	B2B subscription payments
GET	`/transactions/subscription/export`	CSV export

© 2026 GlowTap

Confidential

Subscription Plans

Method	Endpoint	Description
GET	`/plans`	List all plans
POST	`/plans`	Create plan
PATCH	`/plans/:id`	Update plan
GET	`/plans/overview`	Salons per plan aggregation
POST	`/plans/:id/discounts`	Add contract discount
PATCH	`/plans/:id/discounts/:discountId`	Update discount

Reports

Method	Endpoint	Description
GET	`/reports/defaulters`	Daily defaulter report
GET	`/reports/revenue`	Revenue summary (subscription + commission)
GET	`/reports/commission`	Commission earnings by salon, period, region
GET	`/reports/salon-growth`	Onboarding metrics
GET	`/reports/churn`	Churn analysis
POST	`/reports/:type/export`	Export report

© 2026 GlowTap

Confidential

User Access Control

Method	Endpoint	Description
GET	`/users/salon-admins`	List salon admins
GET	`/users/salon-admins/:id`	Detail + branch access
PATCH	`/users/salon-admins/:id`	Update profile
POST	`/users/salon-admins/:id/reset-password`	Trigger reset
POST	`/users/salon-admins/:id/disable`	Disable account
GET	`/users/sub-admins`	List (read-only)
POST	`/users/sub-admins/:id/disable`	Disable only
GET	`/users/end-users`	List end users
POST	`/users/end-users/:id/disable`	Disable account

CMS & Banners

Method	Endpoint	Description
GET/POST/PATCH/DELETE	`/cms/pages`	Website content CRUD
GET/POST/PATCH/DELETE	`/cms/media`	Media library
GET/POST/PATCH/DELETE	`/banners`	In-app banner CRUD
POST	`/banners/:id/preview`	Preview targeting for org

Support Tickets

Method	Endpoint	Description
GET	`/tickets`	List with filters
POST	`/tickets`	Create ticket
GET	`/tickets/:id`	Detail with history
PATCH	`/tickets/:id/status`	Update status
POST	`/tickets/:id/notes`	Add internal note
POST	`/tickets/:id/assign`	Assign to agent

© 2026 GlowTap

Confidential

Analytics

Method	Endpoint	Description
GET	`/analytics/dashboard`	KPI summary incl. subscription + commission revenue
GET	`/analytics/revenue-trend`	Time-series MRR
GET	`/analytics/funnel`	Conversion funnel
GET	`/analytics/feature-adoption`	Module usage

Configuration

Method	Endpoint	Description
GET/POST/PATCH	`/config/reason-codes`	Reason code CRUD
GET/PATCH	`/config/notifications`	Email/SMS templates
GET/PATCH	`/config/settings`	Platform settings
GET	`/audit-logs`	Audit trail with filters

5. Banner Targeting Engine

5.1 Rule Schema

```
{
  "operator": "AND",
  "conditions": [
    { "field": "current_plan_code", "op": "eq", "value": "STARTER_1M" },
    { "field": "subscription_age_days", "op": "gte", "value": 90 },
    { "field": "days_to_expiry", "op": "gt", "value": 30 }
  ]
}
```

5.2 Supported Condition Fields

Field	Type	Description
`current_plan_code`	string	Active plan code
`days_to_expiry`	int	Days until subscription end
`subscription_age_days`	int	Days since subscription start
`active_branch_count`	int	Current active branches
`branch_limit_reached`	boolean	At max branches for plan
`org_status`	string	ACTIVE, SUSPENDED, etc.
`has_pending_renewal`	boolean	Renewal invoice generated
`previous_plan_code`	string	For win-back campaigns

5.3 Evaluation Flow

```
Salon Admin loads dashboard
        |
        ▼
GET /api/v1/salon/banners
        |
        ▼
Banner Service fetches active banners (cached in Redis/Valkey, TTL 5min)
        |
        ▼
For each banner: evaluate targeting_rules against org context
        |
        ▼
Sort by display_priority ASC
        |
        ▼
Return matched banners (max 3 displayed)
```

5.4 Renewal Incentive Logic (Example)

```
function getRenewalBonusBanner(org: OrgContext): Banner | null {
  const { currentPlan, daysToExpiry, targetPlan } = org;

  if (daysToExpiry > 30) return null;

  const bonuses = {
    '12M': { renewSame: 1, upgradeTo24M: 2 },  // free months
    '6M':  { renewSame: 0, upgradeTo12M: 1 },
  };

  const bonus = bonuses[currentPlan.duration];
  if (!bonus) return null;

  // Return appropriate banner based on upgrade vs same-plan renewal
}
```

6. Defaulter Report Engine

6.1 Cron Job Design

```
Schedule: Daily at 06:00 IST (configurable)
Job: generate_defaulter_report

Steps:
1. Query organizations WHERE:
   - subscription.status = 'PAST_DUE' OR
   - subscription.next_bill_date + grace_period_days < TODAY
   - org.status = 'ACTIVE'
2. Enrich with owner contact, outstanding amount, days overdue
3. Store snapshot in defaulter_report_snapshots table
4. Send email to ops distribution list
5. Expose via GET /reports/defaulters?date=YYYY-MM-DD
```

6.2 Defaulter Query (PostgreSQL)

```sql
SELECT
  o.id, o.name, o.owner_name, o.owner_email, o.owner_phone,
  sp.name AS plan_name,
  os.end_date, os.next_bill_date,
  CURRENT_DATE - os.next_bill_date AS days_overdue,
  COALESCE(SUM(st.net_amount) FILTER (WHERE st.payment_status = 'FAILED'), 0) AS outstanding,
  COUNT(b.id) FILTER (WHERE b.status = 'ACTIVE') AS active_branches
FROM organizations o
JOIN organization_subscriptions os ON os.organization_id = o.id AND os.status IN ('ACTIVE','PAST_DUE')
JOIN subscription_plans sp ON sp.id = os.plan_id
LEFT JOIN subscription_transactions st ON st.organization_id = o.id
LEFT JOIN branches b ON b.organization_id = o.id
WHERE o.status = 'ACTIVE'
  AND os.next_bill_date < CURRENT_DATE - :grace_period_days
GROUP BY o.id, sp.name, os.end_date, os.next_bill_date
ORDER BY days_overdue DESC;
```

6.3 No Auto-Suspend Implementation

- Remove any cron/scheduler that auto-suspends on payment failure
- Payment webhook handler sets subscription status to `PAST_DUE` only
- Suspension is exclusively via `POST /organizations/:id/suspend` by Super Admin
- Optional: after N days on defaulter report, auto-create a support ticket for follow-up

---

7. Authentication & Authorization

7.1 Super Admin RBAC

```json
{
  "roles": {
    "SUPER_ADMIN": ["*"],
    "SUPER_ADMIN_READ_ONLY": ["read:*"],
    "OPS_AGENT": [
      "read:leads", "write:leads.approve", "write:leads.reject",
      "read:branch-requests", "write:branch-requests.approve",
      "read:tickets", "write:tickets",
      "read:reports.defaulters",
      "read:organizations"
    ]
  }
}
```

7.2 Security Requirements

- Password policy: min 12 chars, complexity rules
- MFA mandatory for SUPER_ADMIN role
- Session timeout: 30 minutes idle, 8 hours absolute
- IP allowlisting (optional, for production)
- All write operations require audit log entry
- Rate limiting: 100 req/min per user on API gateway
- CSRF protection on web portal
- Sensitive fields (passwords, tokens) never logged

---

8. Event-Driven Integration

8.1 Domain Events

Event	Payload	Consumers
`salon.approved`	org_id, admin_user_id	Email service, Analytics
`salon.suspended`	org_id, reason_code	Salon portal (lock), Email
`branch.approved`	branch_id, org_id	Salon portal, Analytics
`subscription.past_due`	org_id, amount	Defaulter report job
`subscription.renewed`	org_id, plan_id	Banner engine cache invalidation
`ticket.created`	ticket_id, priority	SLA timer, Email notification
`user.disabled`	user_id, user_type	Identity service (revoke sessions)

8.2 Event Bus

- **Job dispatch:** BullMQ queues on Redis/Valkey for async processing (approval pipeline, emails, reports)
- **Domain events:** Emitted from Fastify services after DB commit; consumed by BullMQ workers
- **Inter-service (future):** Redis Streams or AWS SNS/SQS if services are split beyond monolith

---

9. Salon Admin Auto-Provisioning

On salon approval, the Identity Service executes:

```
async function provisionSalonAdmin(org: Organization): Promise<SalonAdmin> {
  const tempPassword = generateSecurePassword(16);

  const user = await identityService.createUser({
    email: org.owner_email,
    name: org.owner_name,
    role: 'SALON_ADMIN',
    organization_id: org.id,
    permissions: {
      branches: 'ALL',  // all current and future branches
      modules: ['appointments', 'manpower', 'services', 'settings', 'billing']
    },
    must_change_password: true
  });

  await emailService.send({
    template: 'SALON_ADMIN_WELCOME',
    to: org.owner_email,
    data: {
      salon_name: org.name,
      login_url: `${SALON_PORTAL_URL}/login`,
      email: org.owner_email,
      temp_password: tempPassword
    }
  });

  return user;
}
```

Sub-admin creation remains exclusively in the Salon Admin portal; Super Admin API exposes read + disable only.

10. Frontend Module Structure

10.1 Super Admin Portal (React + TypeScript)

```
src/
├── app/                          # Routes (React Router)
│   ├── auth/Login.tsx
│   ├── dashboard/Dashboard.tsx
│   ├── salon-approvals/
│   ├── branch-approvals/
│   ├── salons/
│   ├── transactions/
│   ├── subscriptions/
│   ├── reports/
│   ├── users/
│   ├── cms/
│   ├── banners/
│   ├── analytics/
│   ├── support/
│   ├── config/
│   └── audit-log/
├── components/
│   ├── layout/Sidebar.tsx
│   ├── salons/SuspendDialog.tsx
│   ├── salons/EarningsPanel.tsx
│   ├── salons/CommissionEditForm.tsx
│   ├── subscriptions/DiscountForm.tsx
│   └── banners/TargetingRuleBuilder.tsx
├── store/
│   ├── index.ts                  # configureStore
│   ├── slices/
│   │   ├── authSlice.ts
│   │   ├── uiSlice.ts
│   │   └── filtersSlice.ts
│   └── api/
│       ├── baseApi.ts            # RTK Query createApi, baseQuery with auth header
│       ├── organizationsApi.ts
│       ├── subscriptionsApi.ts
│       ├── ticketsApi.ts
│       └── bannersApi.ts
├── hooks/
└── types/
```

10.2 React Native (End User / Salon Mobile)

```
src/
├── screens/
├── components/
├── store/                        # Same RTK Query api slices where shared
│   ├── api/baseApi.ts            # Points to same Fastify backend
│   └── slices/
└── navigation/
```

- Shared `@glowtap/api` package (optional) for RTK Query endpoint definitions consumed by both web and mobile

11. Caching Strategy

Data	Cache (Redis/Valkey)	TTL	Invalidation
Subscription plans	Redis/Valkey	1 hour	On plan CRUD
Reason codes	Redis/Valkey	1 hour	On config change
Banner rules	Redis/Valkey	5 min	On banner CRUD
Analytics KPIs	Redis/Valkey	15 min	Scheduled refresh
User sessions	Redis/Valkey	Session lifetime	On logout/disable
BullMQ job state	Redis/Valkey	Job lifetime	Auto-managed by BullMQ

12. Testing Strategy

Layer	Approach
Unit	Service logic, banner rule evaluator, branch limit calculator
Integration	API endpoints with test DB, approval pipeline
E2E	Playwright: approve salon → verify admin email → login
Load	k6: transaction list with 1M records, report generation
Security	OWASP ZAP scan, RBAC permission matrix tests

12.1 Critical Test Scenarios

1. Approve salon → org + branch + admin + subscription created atomically
2. Branch request at plan limit → rejected with UPGRADE_REQUIRED
3. Downgrade plan with excess branches → blocked
4. Apply RC discount → reflected on next invoice generation
5. Suspend salon → salon portal returns 403, end-user booking blocked
6. Disable sub-admin → active sessions revoked within 60 seconds
7. Banner targeting → correct banners for plan/expiry combinations

8. Defaulter report → includes all past-due orgs after grace period
9. No auto-suspend → payment failure does not change org.status
10. Audit log → every write action recorded with before/after state
11. Commission on txn → correct amount at salon's current rate; stored on transaction
12. Commission rate change → applies to new txns only; history table updated
13. Earnings endpoint → subscription + commission totals match line items

---

13. Deployment & Infrastructure

```
┌──────────────────────────────────────────────────┐
│              Production Environment (AWS)        │
├──────────────────────────────────────────────────┤
│  AWS EC2                  → Fastify API + BullMQ workers │
│  AWS S3                   → Static assets, media, exports│
│  AWS S3 + CloudFront      → React web app (optional CDN) │
│  PostgreSQL or MongoDB    → Primary DB (TBD)             │
│  Redis / Valkey           → Cache + BullMQ backing store │
│  AWS SES / SendGrid       → Email                        │
│  Razorpay / Stripe        → Payments                     │
│  Pino → Cloud Logs        → Monitoring (provider TBD)    │
└──────────────────────────────────────────────────┘
```

13.1 EC2 Production Layout

Process	EC2 Role	Notes
`fastify-api`	API server	PM2 or systemd; behind ALB/nginx reverse proxy
`bullmq-worker`	Job consumer	Same or separate EC2 instance; processes report/email queues
React web build	S3 static hosting or EC2 nginx	Super Admin + Salon Admin portals

13.2 Operations

- **Environments:** dev, staging, production (separate EC2 instances or VPC isolation)
- **CI/CD:** GitHub Actions — lint, test, build, deploy to EC2 via SSH/SSM or CodeDeploy
- **Logging:** Pino structured JSON → cloud log service *(CloudWatch, Datadog, or similar — TBD by Yash)*
- **Monitoring:** Log-based alerts on error rate, job failures, API latency; health check endpoint `/health`
- **Backups:** DB-native backups (pg_dump or MongoDB Atlas/backup cron); S3 lifecycle policies for exports
- **Secrets:** AWS Secrets Manager or SSM Parameter Store on EC2

---

14. Additional Technical Recommendations

14.1 Not Covered in Functional Spec — Technical Additions

Capability	Implementation Notes
Idempotency	`Idempotency-Key` header on approve/reject/charge endpoints
Webhook replay	Store raw payment webhooks; replay on processing failure
Soft delete	Organizations terminated, not deleted; GDPR deletion as separate workflow
Multi-tenancy isolation	Row-level security on org_id for salon-facing APIs
Invoice generation	PDF via Puppeteer/wkhtmltopdf; store in S3, link in subscription_transactions
Feature flags	LaunchDarkly or DB-driven flags for gradual rollouts
Search	PostgreSQL full-text or MongoDB text index; Elasticsearch if needed at scale
Real-time updates	WebSocket/SSE for ticket status and approval queue counts on dashboard
Data retention job	Archive audit logs > 2 years to cold storage
API versioning	`/api/v1/` with deprecation headers

14.2 Performance Targets

Operation	Target
Dashboard load	< 2s
Transaction list (filtered)	< 1s for 20 records
Defaulter report generation	< 30s for 10K salons
Salon approval pipeline	< 5s (async email)
Banner evaluation	< 100ms (cached)

15. Implementation Phases

Phase 1 — Foundation (Weeks 1–4)

- Auth, RBAC, audit log
- Organization & branch data model

- Salon approval + auto-provisioning
- Branch approval with limit check
- Basic All Salons view (suspend/resume/terminate)

Phase 2 — Billing & Subscriptions (Weeks 5–8)

- Subscription plan CRUD
- Contract discounts
- Manual discount (RC/NRC)
- Subscription transactions
- Defaulter report (daily cron)

Phase 3 — Operations (Weeks 9–12)

- End-user transaction history
- User access control (3 tabs)
- Support ticketing
- Reports suite
- Master configuration (reason codes)

Phase 4 — Growth & Engagement (Weeks 13–16)

- CMS for website
- Banner management with targeting engine
- Platform analytics dashboards
- Invoice generation
- Export capabilities

© 2026 GlowTap

Confidential

16. Open Questions & Decisions

#	Question	Options	Status
1	**Database**	PostgreSQL vs MongoDB	**TBD — Yash**
2	**Cloud logging provider**	CloudWatch, Datadog, Grafana Loki, etc.	**TBD — Yash**
3	Branch limit on downgrade	Block vs auto-deactivate oldest branches	**Block** — require salon admin to choose
4	Grandfathering on plan changes	Existing salons keep old limits vs force migration	**Grandfather** with migration prompt at renewal
5	Sub-admin disable notification	Notify salon admin only vs also sub-admin	**Both**
6	Defaulter grace period	Fixed days vs configurable per plan	**Configurable** in platform settings (default 7 days)
7	Payment retry	Auto-retry failed payments before PAST_DUE	**Yes** — 3 retries over 5 days, then PAST_DUE
8	Multi-currency	Single currency vs multi	**Single (INR)** for MVP; schema supports currency field
9	KYC requirement	Mandatory vs optional at approval	**Optional** Phase 1; mandatory for enterprise
10	Banner A/B testing	Phase 1 vs later	**Later** — Phase 4 enhancement
11	Redis vs Valkey	Managed Redis (ElastiCache) vs Valkey	Either; API-compatible, choose based on AWS pricing

17. Document Control

Version	Date	Author	Changes
1.2	May 31, 2026	GlowTap Engineering	Commission model, earnings APIs, org commission_pct schema
1.1	May 31, 2026	GlowTap Engineering	Confirmed stack: React/RN, RTK+RTK Query, Fastify, Redis/Valkey, S3, BullMQ, EC2, Pino
1.0	May 31, 2026	GlowTap Engineering	Initial release

© 2026 GlowTap

Confidential

Companion document: Super Admin Functional Specification (non-technical).